MUD 게임 보고서

C++프로그래밍 및 실습 | project/week10/mud_game.cpp

학번 : 242324 이름 : 김민서

1. 서론

1.1 프로젝트 목적 및 배경

7주차까지 배운 2차원 배열, 조건문, 반복문, 함수, 참조 변수 내용을 실습하기 위해 진행하였다.

1.2 목표

2차원 배열로 만든 5x5 지도 위에서 유저가 상하좌우로 이동하며 목적지에 도착하는 간단한 MUD 게임을 구현한다.

2. 요구사항

2.1 사용자 요구사항

유저가 텍스트 명령어로 상하좌우로만 이동하며, 체력(HP)이 0이 되기 전에 목적지에 도착하는 게임

2.2 기능 계획

[기본 기능]

- ① 사용자에게 "상", "하", "좌", "우", "지도", "종료" 중 하나를 입력받기
  - 상/하/좌/우 입력 시 해당 방향으로 이동 후 지도 출력
  - "지도"를 입력하면 전체 지도와 함께 현재 위치를 출력
  - 이 중 다른 것을 입력하면 에러 메시지 출력 후 재입력 요청
- ② 지도 밖으로 나가게 되면 에러 메시지 출력
- ③ 목적지에 도착하면 "성공"을 출력하고 종료

[추가 기능]

- ① 유저는 체력 20을 가지고 게임 시작
- ② 사용자가 이동할 때마다 체력 1씩 감소
- ③ 명령어를 입력받을 때마다 HP 함께 출력

- ④ HP가 0이 되면 "실패"를 출력하고 종료
- ⑤ 아이템, 포션, 적을 만났을 때 그에 대한 메시지 출력
  - 적을 만나면 HP 2 감소, 포션을 만나면 HP 2 증가 (적과 포션은 사라지지 않음)

2.3 함수 계획

- ① main() : 사용자에게 명령어를 계속 입력받고, 그에 맞는 함수 호출
- ② displayMap() : 지도와 현재 위치 출력
- ③ checkXY() : 이동하려는 좌표가 지도 안인지 체크
- ④ checkGoal() : 목적지에 도착했는지 체크
- ⑤ checkOutOfMap() : 지도 밖으로 나갔으면 메시지를 출력하고 원래 자리로 되돌림
- ⑥ decreaseHP() : 이동할 때마다 HP 1 감소
- ⑦ checkState() : 이동한 칸의 아이템/적/포션을 확인하고 메시지 출력 및 HP 변경

3. 설계 및 구현

3.1 기능 별 구현 사항

(1) 전역 변수와 지도 초기화

```
// 맵의 가로, 세로 크기
const int mapX = 5;
const int mapY = 5;
int currentHP = 20;      // 유저의 현재 HP

// 0은 빈 공간, 1은 아이템, 2는 적, 3은 포션, 4는 목적지
int map[mapY][mapX] = { {0, 1, 2, 0, 4},
                        {1, 0, 0, 2, 0},
                        {0, 0, 0, 0, 0},
                        {0, 2, 3, 0, 0},
                        {3, 0, 0, 0, 2} };


// 유저의 위치를 저장할 변수
int user_x = 0; // 가로 번호
int user_y = 0; // 세로 번호
```

입력	없음
반환값	없음
결과	5x5 지도, 유저 시작 위치 (0, 0), HP 20이 준비됨
설명	1. mapX, mapY를 const 전역 상수로 선언하여 지도의 가로, 세로 크기를 5로 고정한다.

	2. currentHP를 전역 변수로 선언하여 main()과 checkState()에서 같은 HP를 사용한다. 3. 2차원 배열 map에 0(빈 공간), 1(아이템), 2(적), 3(포션), 4(목적지)를 미리 저장한다. 4. user_x(가로), user_y(세로)를 0으로 초기화하여 왼쪽 위에서 시작한다.

(2) main() - 명령어 입력 및 처리

```
32      // 게임 시작
33      while (1) { // 사용자에게 계속 입력받기 위해 무한 루프
34
35          // 사용자의 입력을 저장할 변수
36          string user_input = "";
37
38          cout << "현재 HP : " << currentHP << " " << "명령어를 입력하세요 (상, 하, 좌, 우, 지도, 종료) : ";
39          cin >> user_input;
40
41          // 이동 명령어 처리
42          if (user_input == "상") {
43              // 위로 한 칸 올라가기
44              user_y -= 1;
45              if (!checkOutOfMap(user_x, mapX, user_y, mapY)) {   // 맵 안이면 이동 성공
46                  cout << "위로 한 칸 올라갑니다." << endl;
47                  decreaseHP(currentHP);
48                  displayMap(map, user_x, user_y);
49                  checkState(map, user_x, user_y);
50              }
51          }
52          else if (user_input == "하") {
53              // 아래로 한 칸 내려가기
54              user_y += 1;
55              if (!checkOutOfMap(user_x, mapX, user_y, mapY)) {
56                  cout << "아래로 한 칸 내려갑니다." << endl;
57                  decreaseHP(currentHP);
58                  displayMap(map, user_x, user_y);
59                  checkState(map, user_x, user_y);
60              }
61          }
62          else if (user_input == "좌") {
63              // 왼쪽으로 이동하기
64              user_x -= 1;
65              if (!checkOutOfMap(user_x, mapX, user_y, mapY)) {
66                  cout << "왼쪽으로 이동합니다." << endl;
67                  decreaseHP(currentHP);
68                  displayMap(map, user_x, user_y);
69                  checkState(map, user_x, user_y);
70              }
71          }
72          else if (user_input == "우") {
73              // 오른쪽으로 이동하기
74              user_x += 1;
75              if (!checkOutOfMap(user_x, mapX, user_y, mapY)) {
76                  cout << "오른쪽으로 이동합니다." << endl;
77                  decreaseHP(currentHP);
78                  displayMap(map, user_x, user_y);
79                  checkState(map, user_x, user_y);
80              }
81          }
```



```
81          }
82          else if (user_input == "지도") {
83              // 지도 보여주기 함수 호출 (이동하지 않으므로 HP 변화 없음)
84              displayMap(map, user_x, user_y);
85          }
86          else if (user_input == "종료") {
87              cout << "종료합니다.";
88              break;
89          }
90          else {
91              cout << "잘못된 입력입니다." << endl;
92              continue;
93          }
```

입력	user_input = 사용자가 입력한 명령어 currentHP = 현재 HP
반환값	0 (프로그램 정상 종료)
결과	명령어에 따라 이동, 지도 출력, 종료, 에러 메시지 출력 중 하나를 수행
설명	1. while(1) 무한 루프로 명령어를 계속 입력받는다. 2. 입력받기 전에 "현재 HP : 20"처럼 현재 HP를 함께 출력한다. 3. 상/하/좌/우를 입력하면 user_x 또는 user_y를 1 바꾼 뒤 checkOutOfMap()으로 지도 안인지 확인한다. 4. 지도 안이면 이동 메시지 출력 → decreaseHP()로 HP 1 감소 → displayMap()으로 지도 출력 → checkState()로 이동한 칸 확인 순서로 실행한다. 5. "지도"를 입력하면 이동 없이 displayMap()만 호출한다. (HP 변화 없음) 6. "종료"를 입력하면 break로 반복문을 빠져나간다. 7. 그 외의 입력은 "잘못된 입력입니다."를 출력하고 continue로 다시 입력받는다.

(3) main() - 실패/성공 체크

```
95          // HP가 0 이하가 되면 실패 후 종료
96          if (currentHP <= 0) {
97              cout << "HP가 0 이하가 되었습니다. 실패했습니다." << endl;
98              cout << "게임을 종료합니다." << endl;
99              break;
100         }
101
102         // 목적지에 도달했는지 체크
103         bool finish = checkGoal(map, user_x, user_y);
104         if (finish == true) {
105             cout << "목적지에 도착했습니다! 축하합니다!" << endl;
106             cout << "게임을 종료합니다." << endl;
107             break;
108         }
```

입력	currentHP = 현재 HP map = 전체 지도 user_x, user_y = 현재 위치
반환값	없음
결과	HP가 0 이하이면 실패 메시지, 목적지이면 성공 메시지를 출력하고 게임 종료
설명	1. 이동과 적/포션 처리가 모두 끝난 뒤 currentHP가 0 이하인지 if로 체크한다. 0 이하이면 "HP가 0 이하가 되었습니다. 실패했습니다."를 출력하고 break로 종료한다. 2. checkGoal()이 true를 반환하면 "목적지에 도착했습니다! 축하합니다!"를 출력하고 break로 종료한다.

(4) displayMap() - 지도와 현재 위치 출력

```cpp
// 지도와 사용자 위치 출력하는 함수
void displayMap(int map[][mapX], int user_x, int user_y) {
    for (int i = 0; i < mapY; i++) {
        for (int j = 0; j < mapX; j++) {
            if (i == user_y && j == user_x) {
                cout << " USER |"; // 양 옆 1칸 공백
            }
            else {
                int posState = map[i][j];
                switch (posState) {
                case 0:
                    cout << "      |"; // 6칸 공백
                    break;
                case 1:
                    cout << "아이템|";
                    break;
                case 2:
                    cout << "  적  |"; // 양 옆 2칸 공백
                    break;
                case 3:
                    cout << " 포션 |"; // 양 옆 1칸 공백
                    break;
                case 4:
                    cout << "목적지|";
                    break;
                }
            }
        }
        cout << endl;
        cout << " ------------------------------ " << endl;
    }
```

입력	int map[][] = 전체 지도 user_x = 유저 x값 user_y = 유저 y값
반환값	없음 (void)
결과	전체 지도와 사용자 위치(USER)를 출력
설명	1. 이중 for문으로 2차원 배열에 있는 지도를 한 칸씩 출력한다. 2. 출력하다가 사용자 위치와 같은 좌표를 만나면 "USER"를 출력한다. 3. 그 외의 칸은 switch문으로 값(0~4)에 따라 빈칸, 아이템, 적, 포션, 목적지를 출력한다. 4. 한 줄 출력이 끝나면 구분선을 출력한다.

(5) checkXY() - 좌표 유효성 체크

```
147  // 이동하려는 곳이 유효한 좌표인지 체크하는 함수
148  bool checkXY(int user_x, int mapX, int user_y, int mapY) {
149      bool checkFlag = false;
150      if (user_x >= 0 && user_x < mapX && user_y >= 0 && user_y < mapY) {
151          checkFlag = true;
152      }
153      return checkFlag;
154  }
155
```

입력	user_x, user_y = 이동하려는 좌표 mapX, mapY = 지도의 가로, 세로 크기
반환값	bool (지도 안이면 true, 밖이면 false)
결과	이동하려는 좌표가 지도 안인지 알려줌
설명	user_x가 0 이상 mapX 미만이고, user_y가 0 이상 mapY 미만이면 true를 반환한다.

(6) checkGoal() - 목적지 도착 체크

```
156  // 유저의 위치가 목적지인지 체크하는 함수
157  bool checkGoal(int map[][mapX], int user_x, int user_y) {
158      // 목적지 도착하면
159      if (map[user_y][user_x] == 4) {
160          return true;
161      }
162      return false;
163  }
```

입력	int map[][] = 전체 지도 user_x, user_y = 현재 위치
반환값	bool (목적지이면 true, 아니면 false)
결과	현재 위치가 목적지인지 알려줌
설명	map[user_y][user_x]의 값이 4(목적지)이면 true를 반환한다.

(7) checkOutOfMap() - 지도 밖 처리

```
// 맵을 벗어났으면 메시지를 출력하고 맵 안으로 되돌리는 함수
bool checkOutOfMap(int& user_x, int mapX, int& user_y, int mapY) {
    bool inMap = checkXY(user_x, mapX, user_y, mapY);
    if (!inMap) {
        cout << "맵을 벗어났습니다. 다시 돌아갑니다." << endl;
        user_x = (user_x < 0) ? 0 : (user_x >= mapX) ? mapX - 1 : user_x;
        user_y = (user_y < 0) ? 0 : (user_y >= mapY) ? mapY - 1 : user_y;
    }
    return !inMap;
}
```

입력	user_x, user_y = 이동한 좌표 (참조) mapX, mapY = 지도 크기
반환값	bool (지도 밖이면 true, 지도 안이면 false)
결과	지도 밖이면 "맵을 벗어났습니다. 다시 돌아갑니다."를 출력하고 위치를 지도 안으로 되돌림
설명	1. checkXY()를 호출하여 지도 안인지 확인한다. 2. 지도 밖이면 에러 메시지를 출력하고, 범위를 벗어난 좌표를 지도 끝(0 또는 4)으로 되돌린다. 3. user_x, user_y를 참조(&)로 받기 때문에 되돌린 위치가 main()의 변수에 그대로 반영된다.

(8) decreaseHP() - 이동 시 HP 감소

```
// 사용자가 이동할 때 마다 사용자 체력 1씩 감소하는 함수
void decreaseHP(int& currentHP) {
    currentHP -= 1;
}
```

입력	currentHP = 현재 HP (참조)
반환값	없음 (void)
결과	HP가 1 줄어듦
설명	currentHP를 참조(&)로 받아 1을 빼므로, 함수 밖의 HP 값이 직접 바뀐다.

(9) checkState() - 아이템/적/포션 처리

```cpp
// 유저가 이동한 칸에 무엇이 있는지 확인해서 메시지를 출력하고 HP를 바꾸는 함수
void checkState(int map[][mapX], int user_x, int user_y) {
    switch (map[user_y][user_x]) {
    case 1:     // 아이템
        cout << "아이템이 있습니다." << endl;
        break;
    case 2:     // 적: HP 2 감소
        currentHP -= 2;
        cout << "적이 있습니다. HP가 2 줄어듭니다." << endl;
        break;
    case 3:     // 포션: HP 2 회복
        currentHP += 2;
        cout << "포션이 있습니다. HP가 2 회복됩니다." << endl;
        break;
    }
}
```

입력	int map[][] = 전체 지도 user_x, user_y = 이동한 위치
반환값	없음 (void)
결과	칸에 있는 것에 따라 메시지를 출력하고 전역 변수 currentHP를 바꿈
설명	1. switch문으로 map[user_y][user_x]의 값을 확인한다. 2. 1(아이템)이면 "아이템이 있습니다."를 출력한다. 3. 2(적)이면 HP를 2 줄이고 "적이 있습니다. HP가 2 줄어듭니다."를 출력한다. 4. 3(포션)이면 HP를 2 늘리고 "포션이 있습니다. HP가 2 회복됩니다."를 출력한다. 5. 적과 포션은 지도에서 사라지지 않으므로 map 배열은 바꾸지 않는다.

4. 테스트

4.1 기능 별 테스트 결과

① 이동 및 지도 출력

```
C:\CPP-26-2\project\week10>C:\CPP-26-2\project\week10\mud_game.exe
현재 HP : 20 명령어를 입력하세요 (상, 하, 좌, 우, 지도, 종료) : 우
오른쪽으로 이동합니다.
     | USER |  적  |     |목적지|
 -------------------------------
아이템|      |      |  적  |     |
 -------------------------------
     |      |      |      |     |
 -------------------------------
     |  적  | 포션 |      |     |
 -------------------------------
 포션 |      |      |      |  적  |
 -------------------------------
아이템이 있습니다.
현재 HP : 19 명령어를 입력하세요 (상, 하, 좌, 우, 지도, 종료) :
```

② 지도 밖으로 나가면 에러 메시지 출력

```
현재 HP : 19 명령어를 입력하세요 (상, 하, 좌, 우, 지도, 종료) : 상
맵을 벗어났습니다. 다시 돌아갑니다.
```

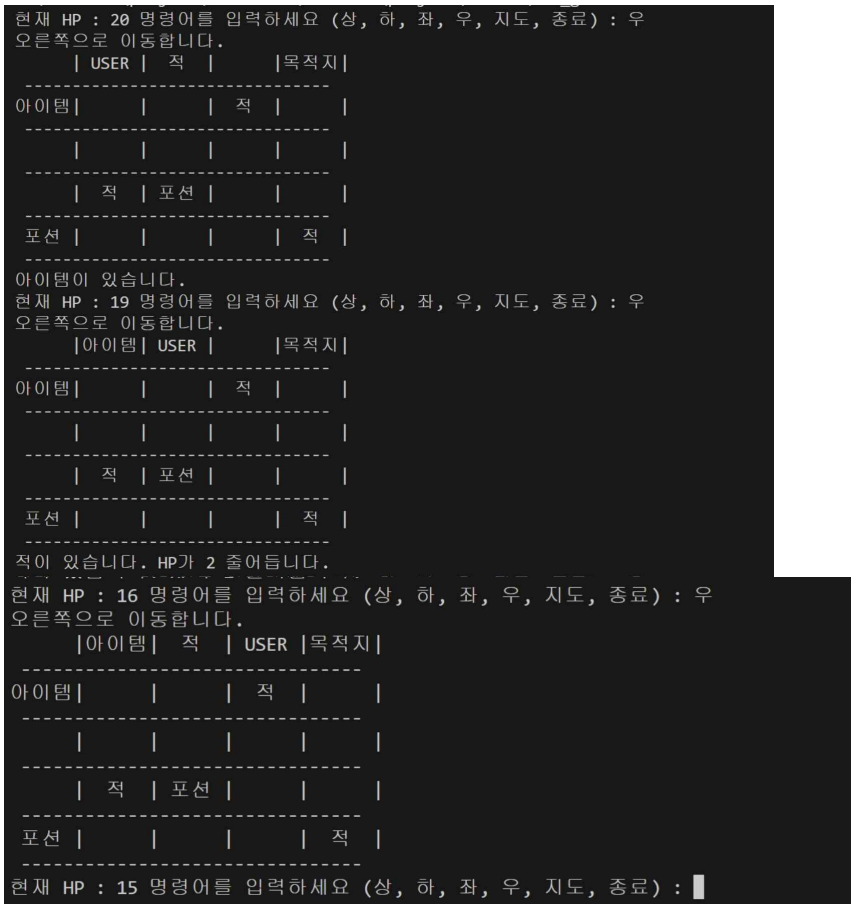
③ "지도"를 입력하면 전체 지도와 현재 위치 출력

```
현재 HP : 19 명령어를 입력하세요 (상, 하, 좌, 우, 지도, 종료) : 지도
     | USER |  적  |     |목적지|
 -------------------------------
아이템|      |      |  적  |     |
 -------------------------------
     |      |      |      |     |
 -------------------------------
     |  적  | 포션 |      |     |
 -------------------------------
 포션 |      |      |      |  적  |
 -------------------------------
현재 HP : 19 명령어를 입력하세요 (상, 하, 좌, 우, 지도, 종료) :
```

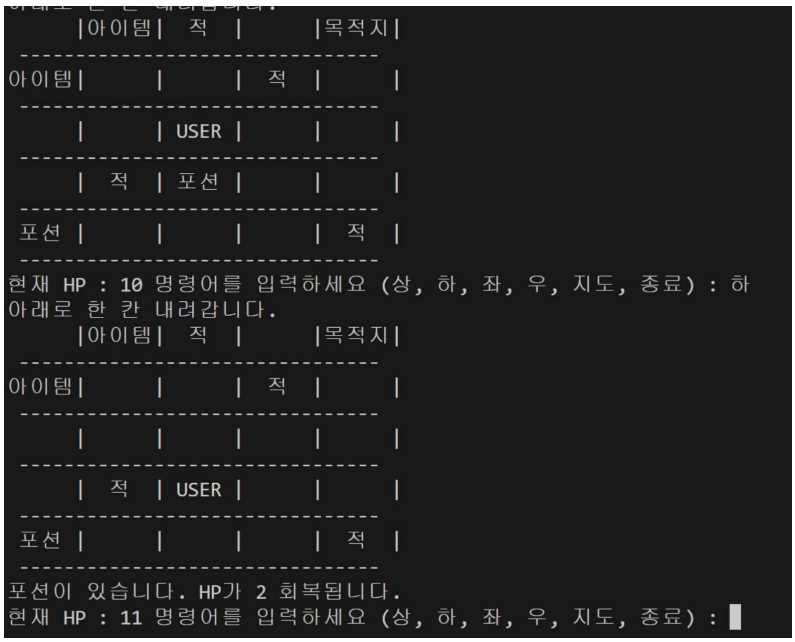
④ 잘못된 명령어 입력 시 에러 메시지 출력

```
현재 HP : 19 명령어를 입력하세요 (상, 하, 좌, 우, 지도, 종료) : 유
잘못된 입력입니다.
```

⑤ 이동할 때마다 HP 1 감소, 아이템/적 메시지 출력



⑥ 포션을 만나면 HP 2 증가



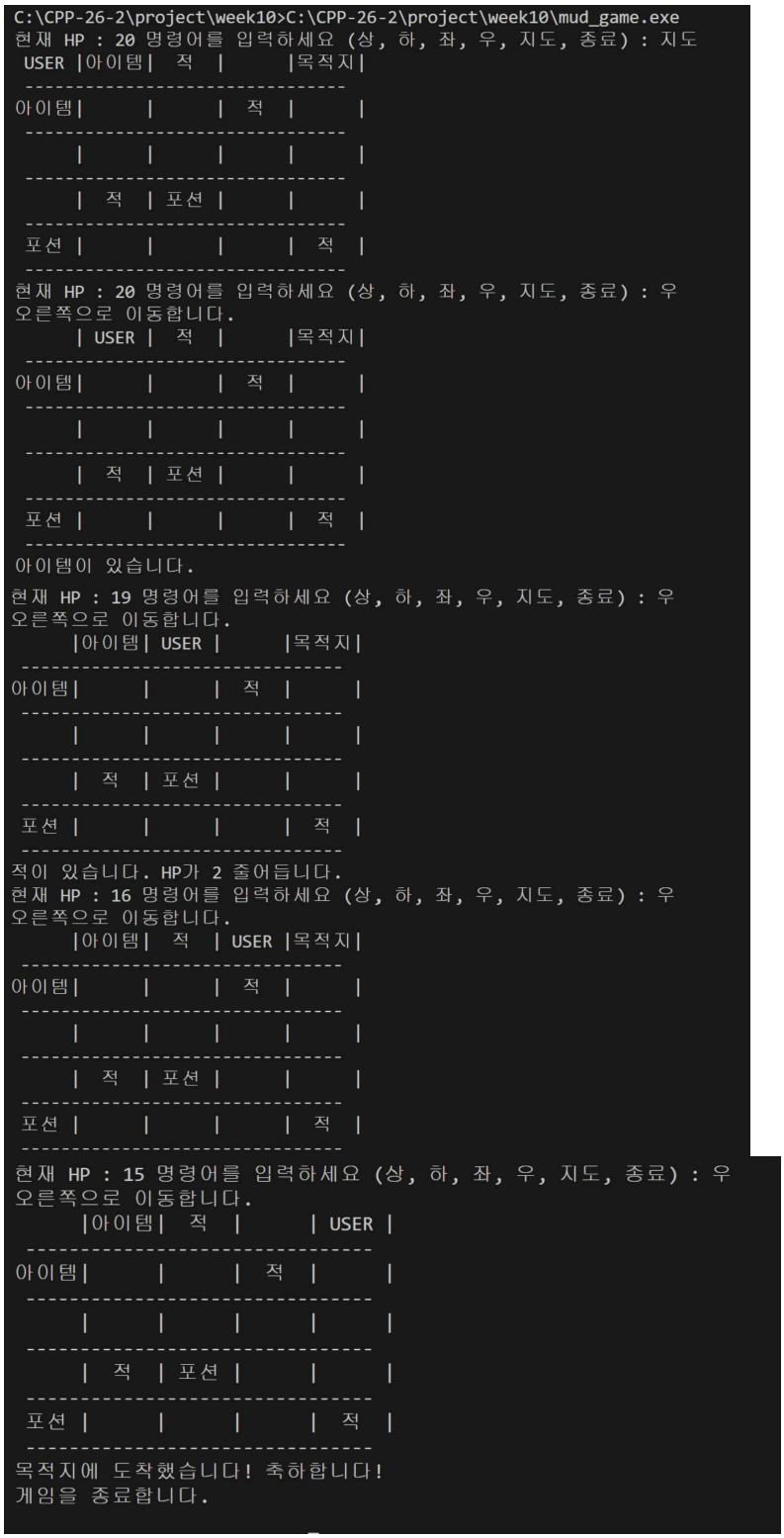
⑦ HP가 0이 되면 실패 출력 후 종료

```
현재 HP : 1 명령어를 입력하세요 (상, 하, 좌, 우, 지도, 종료) : 좌
왼쪽으로 이동합니다.
     |아이템|  적   |      |목적지|
 -------------------------------
아이템|      |      |  적   |      |
 -------------------------------
     |      |      |      |      |
 -------------------------------
     |  적   | 포션 | USER |      |
 -------------------------------
 포션 |      |      |      |  적   |
 -------------------------------
HP가 0 이하가 되었습니다. 실패했습니다.
게임을 종료합니다.
```

⑧ 목적지 도착 시 성공 출력 후 종료

```
현재 HP : 16 명령어를 입력하세요 (상, 하, 좌, 우, 지도, 종료) : 우
오른쪽으로 이동합니다.
     |아이템|  적   | USER |목적지|
 -------------------------------
아이템|      |      |  적   |      |
 -------------------------------
     |      |      |      |      |
 -------------------------------
     |  적   | 포션 |      |      |
 -------------------------------
 포션 |      |      |      |  적   |
 -------------------------------
현재 HP : 15 명령어를 입력하세요 (상, 하, 좌, 우, 지도, 종료) : 우
오른쪽으로 이동합니다.
     |아이템|  적   |      | USER |
 -------------------------------
아이템|      |      |  적   |      |
 -------------------------------
     |      |      |      |      |
 -------------------------------
     |  적   | 포션 |      |      |
 -------------------------------
 포션 |      |      |      |  적   |
 -------------------------------
목적지에 도착했습니다! 축하합니다!
게임을 종료합니다.
```

4.2 최종 테스트 스크린샷



5. 결과 및 결론

5.1 프로젝트 결과

2차원 배열과 함수를 이용하여 HP, 아이템, 적, 포션 기능이 있는 간단한 MUD 게임을 만들었다.